

SortNow: End Term Evaluation Report

Author: Lavanya Nigam

Github Repository: <https://github.com/lavanyanigam/SortNow>

Daily Log:  **Daily Log BYOP: Lavanya Nigam**

Section 1: Abstract

The idea of this project is proper waste segregation at source to support Indian waste management systems and make waste disposal hassle-free. The objective is to develop an artificial intelligence model that helps detect and classify waste items into 6 categories: Biodegradable, Cardboard, Glass, Metal, Paper, and Plastic. This helps users responsibly dispose their waste when faced with multiple bins.

The methodology involved implementing a custom multi-object detection model with a 14×14 grid detection system, trained on 7,260 preprocessed images from the Roboflow Garbage Classification dataset [\[1\]](#). The model used convolutional neural networks with batch normalization, LeakyReLU, and Non-Maximum Suppression, similar to the YOLOv1 architecture for multi-object detection [\[2\]](#).

Training was done on Kaggle GPU P100 using PyTorch with Adam optimizer and the YOLO loss function for multiple objects. The model was trained until epoch 64 because early stopping was triggered, with the best performance at epoch 54 (validation loss: 176.31, training loss: 65.58).

The final deliverable is an interactive and user-friendly Flask web application that runs locally on the user's setup and includes real-time waste classification. Users can upload waste images and get instant predictions with bounding boxes, class labels, confidence scores for prediction, and color-coded bin recommendations for ease of use by the general public. This project demonstrates the practical application of deep learning for environmental sustainability.

Section 2: Methodology

Problem Statement

Despite India's Solid Waste Management Rules 2016 mandating waste segregation at source, citizens struggle to identify correct disposal categories, leading to poor waste disposal. When faced with multiple bins, uncertainty about classifying everyday items like cups, tissues, or wrappers creates friction. This project is an image classification system that eliminates this confusion at the moment of disposal, resulting in responsible waste sorting through instant identification and disposal assistance.

Data Collection and Preprocessing

Data Sources:

The original dataset was obtained from Roboflow's Garbage Classification repository [\[1\]](#), containing over 10,000 annotated images across six waste categories. This dataset contained real-world waste objects in varying lighting conditions, and object arrangements.

Dataset Characteristics:

- **Original size:** 10,000+ images
- **Classes:** 6 (Biodegradable, Cardboard, Glass, Metal, Paper, Plastic)
- **Annotation format:** YOLO format with bounding box coordinates
- **Image resolution:** Variable, standardized to 448×448 during training

Preprocessing Steps:

1. **Dataset Analysis:** Examined dataset class distribution, and quality. Found that some images contained more than 100 objects, which would introduce noise during training.
2. **Quality Filtering:** Removed images with more than 98 objects per image.
3. **Data Cleaning:** Verified image-label pairs, removed corrupted images.
4. **Final Dataset:** After preprocessing, the cleaned dataset contained 7,260 training images and 3,114 validation images, but with unbalanced class distribution.

Data Augmentation: Images normalized to 448×448 resolution

Algorithms and Techniques

Selected Approach: Custom YOLO Architecture

YOLO (You Only Look Once) was chosen because:

1. **Multi-object Detection:** YOLO's grid-based mechanism detects multiple objects in a single image within one pass.
2. **End-to-end Training:** YOLO does both detection and classification, simplifying the training.

Why Not Pre-trained Models: While considered initially, building a custom model provided deeper understanding of object detection fundamentals.

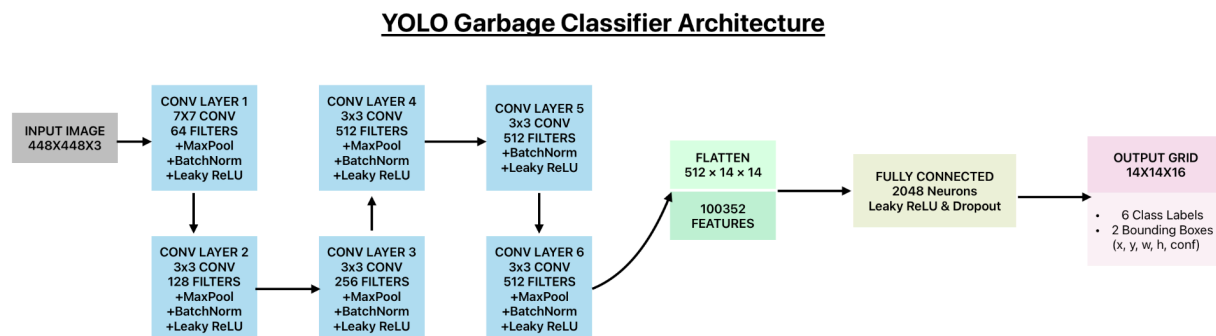
Techniques:

1. **Grid-based Detection:** 14×14 grid (196 cells), with each cell predicting 2 bounding boxes and 6 class probabilities.
2. **Non-Maximum Suppression (NMS):** Filters duplicate detections, keeping only the highest-confidence prediction per object.
3. **Confidence Threshold:** Filters low-confidence predictions (threshold: 0.2 in model)

Model Development

Architecture Details:

- Input Size: 448×448 pixels
- Grid Size: 14×14 (S=14)
- Bounding Boxes per Cell: 2
- Classes: 6 waste categories



Output Structure:

For each grid cell: [C1, C2, C3, C4, C5, C6, Conf1, x1, y1, w1, h1, Conf2, x2, y2, w2, h2]

- C1-C6: Class probabilities for 6 waste categories
- Conf1/Conf2: Confidence scores for 2 bounding boxes
- x, y, w, h: Bounding box coordinates and dimensions

Optimization:

Loss Function: Similar to YOLOv1, my model had:

1. **Localization Loss:** Mean Squared Error for bounding box coordinates (weight: 5.0)
2. **Confidence Loss:** Penalizes incorrect confidence scores
3. **No-Object Loss:** Penalizes false positives (weight: 0.5)
4. **Classification Loss:** Cross-entropy loss (with class-specific weights to address imbalance in dataset)

Class Weights Applied (according to analysis of dataset during preprocessing):

- Biodegradable: 0.136
- Cardboard: 0.884
- Glass: 0.574
- Metal: 0.779
- Paper: 1.000 (reference weight)
- Plastic: 0.771

Optimizer: Adam optimizer (learning rate $1e-4$)

Regularization:

- Dropout (0.5) in fully connected layers
- Batch Normalization
- Early Stopping with patience of 10
- Weight Decay ($1e-4$) through optimizer

Training Strategy:

- Batch size: 8 images per batch
- Maximum epochs: 100
- Early stopping triggered at epoch 64
- Best model saved at epoch 54
- Learning rate reduced when validation loss plateaued

Evaluation Methods:

- Early stopping based on validation loss
- Manual testing on diverse waste images
- Learning rate scheduling

Implementation Workflow

1. Environment Setup

- Installed Flask, MySQL, mysql-connector-python, Jupyter Notebook, Homebrew
- Installed Ultralytics (YOLO), PyTorch, OpenCV, Pillow, Matplotlib
- Initialized local Git repository

2. Dataset Collection & Preprocessing

- Downloaded waste-classification datasets from Kaggle and Roboflow
- Resized and cleaned images using VS Code
- Dataset had 6 classes: BIODEGRADABLE, CARDBOARD, GLASS, METAL, PAPER, PLASTIC
- Balanced dataset by removing images with excessive labels

3. Model Research & Learning

- Studied CNN fundamentals, PyTorch autograd, backpropagation, and gradient descent
- Learned YOLO architecture, labeling format, IoU, and mAP metrics
- Referred to Ultralytics docs, research blogs, and video tutorials

4. YOLO-Based Model Training

- Trained YOLOv8n on a small dataset subset (Colab) for validation
- Trained YOLOv8n and YOLOv8s on the full dataset with tuned hyperparameters
- Trained final YOLOv11s model on Kaggle
- Evaluated performance using IoU and mAP

5. Custom Model Development (From Scratch)

- Implemented CNN-based single-object classifier in PyTorch [\[3\]](#)
- Followed along custom YOLOv1 from YouTube [\[4\]](#)
- Used Conv2D, MaxPooling, and Linear layers
- Applied hyperparameter tuning
- Trained for 100 epochs with varying learning rates and batch sizes

6. Backend & Frontend Integration

- Built Flask backend for model inference
- Developed frontend using HTML and CSS
- Connected trained models to the web application

7. Deployment

- Committed all training and inference code to GitHub (excluding datasets)
 - Organized repository structure and updated README
 - Added training results, made the project reproducible
-

Section 3: Results

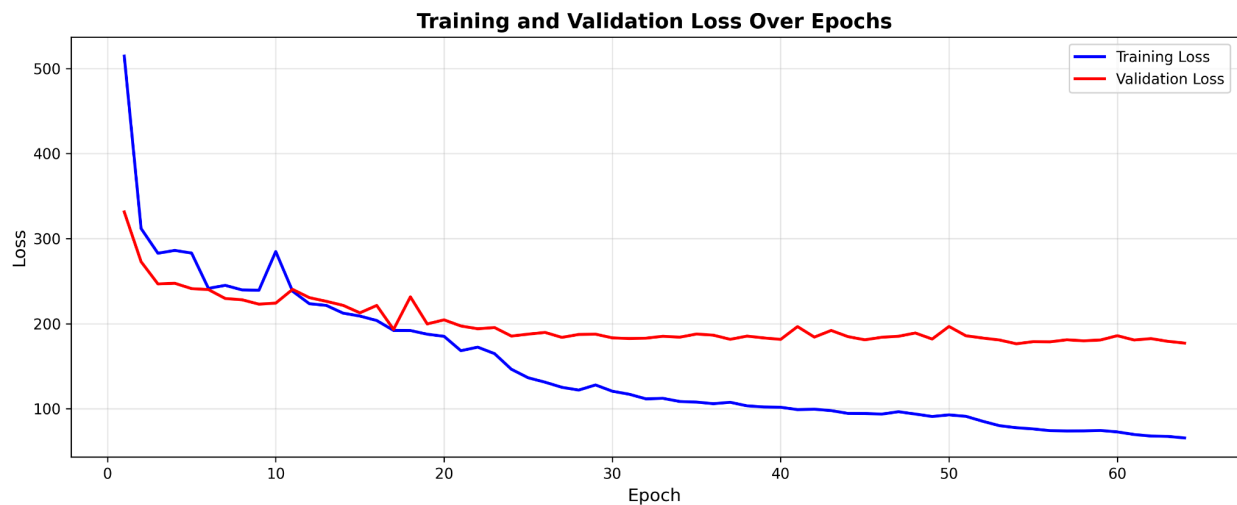
Training Performance:

The model was trained for 100 epochs with an early stopping feature. Training stopped at epoch 64, with the best model performance at epoch 54.

Key Metrics:

- **Best Validation Loss:** 176.31 (Epoch 54)
- **Final Training Loss:** 65.58 (Epoch 64)
- **Final Validation Loss:** 177.08 (Epoch 64)
- **Training Duration:** Approximately 2 hours 25 min on Kaggle GPU P100

Loss Evolution:



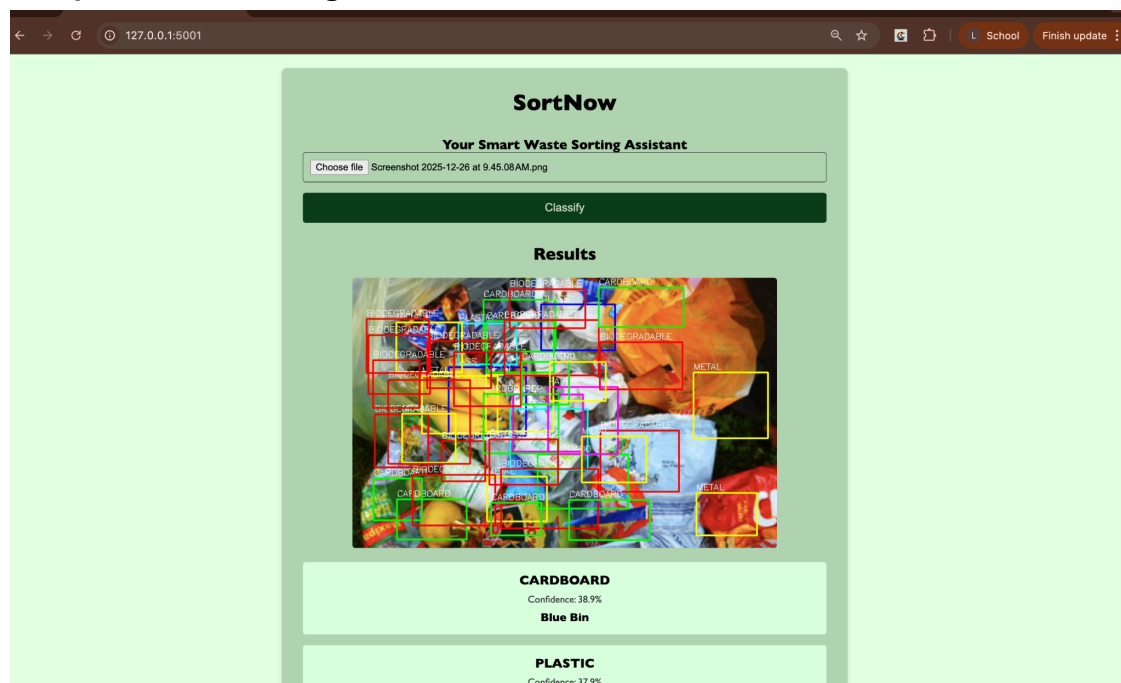
The training graph shows:

- Initial rapid loss reduction in first 10 epochs
- Steady improvement through epochs 10-50
- Early stopping triggered at epoch 64
- Validation loss plateaued

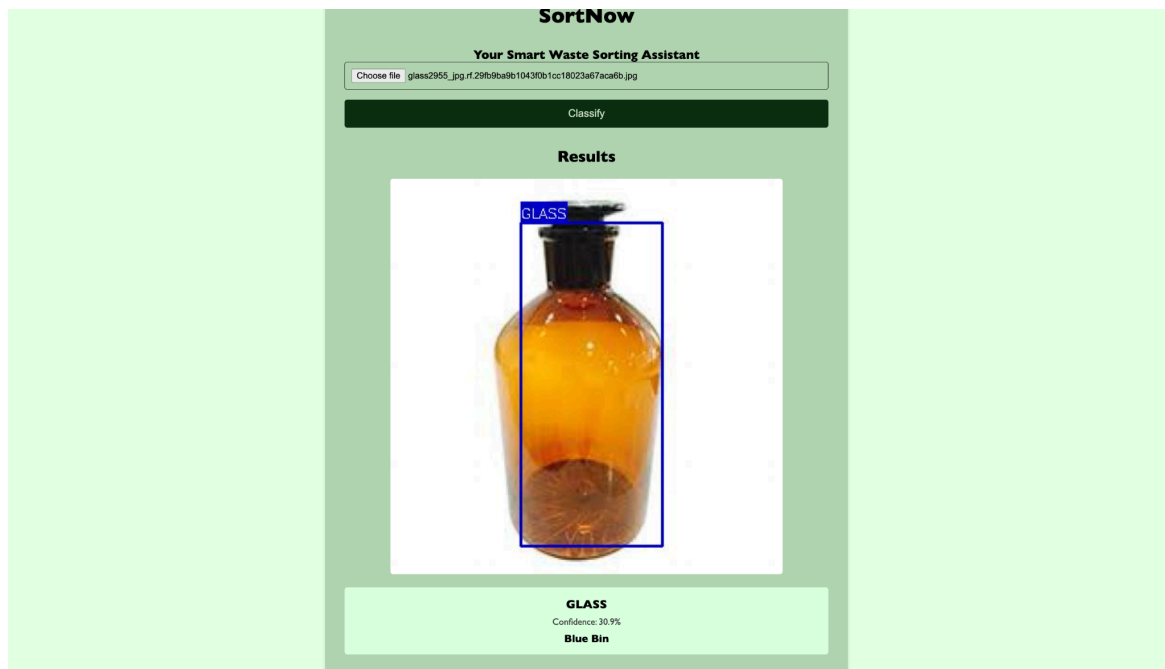
Model Behavior: Performs best on single or moderately grouped waste items with clear separation between objects

1. **Classification Bias:** Exhibits tendency to predict Paper and Cardboard categories more frequently
2. **Multi-object Limitation:** Performance degrades on densely packed scenes with multiple overlapping objects, as training dataset contained sparse arrangements
3. **Training Dataset Limitation:** Reduced accuracy on real-world cluttered waste bin scenarios, because of mismatch between training dataset and practical real-life images.

Sample Success Images:



*all classes not shown in screenshot, can view on scrolling the web page



127.0.0.1:5001

SortNow

Your Smart Waste Sorting Assistant

Choose file 1_j9BnLsdm0LWdsDAcGUSRA.jpg

Classify

Results



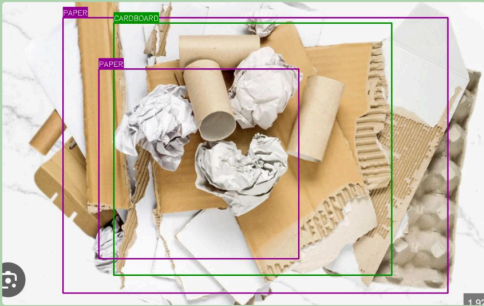
PAPER
Confidence: 29.5%
Blue Bin

CARDBOARD
Confidence: 27.0%
Blue Bin

Screenshot 2026-01-05 at 12:22:27 AM

Classify

Results

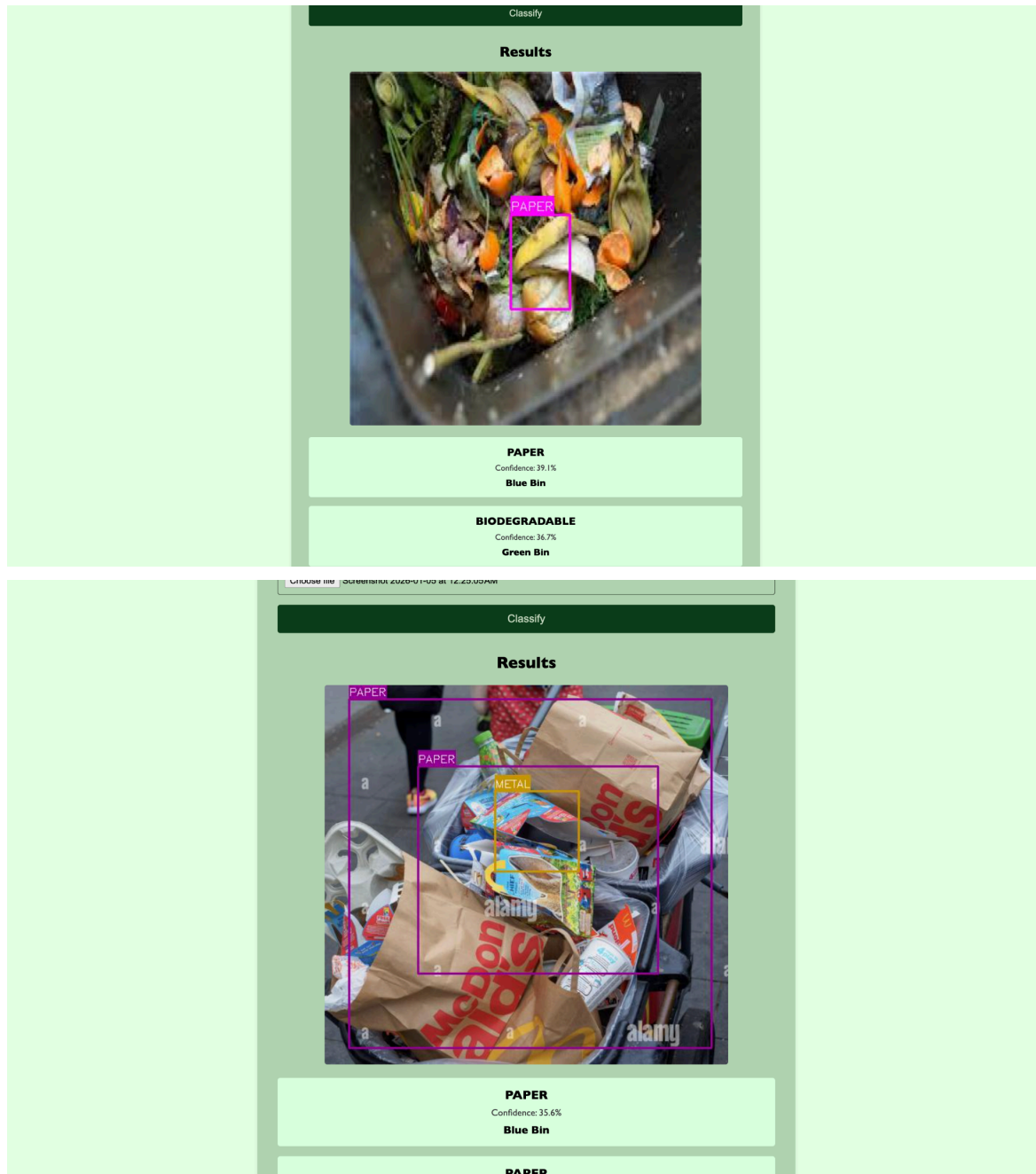


PAPER
Confidence: 34.5%
Blue Bin

CARDBOARD
Confidence: 28.7%
Blue Bin

PAPER
Confidence: 22.6%
Blue Bin

Sample Failure Images:



Section 4: Final Deliverable

- Functional Flask web app displaying bounding boxes on detected waste objects with bin-specific disposal recommendations
- Near-instant inference response (<0.5 seconds per image on CPU) with immediate visual feedback

- Automatic image deletion on page reload eliminates storage overhead and ensures privacy
- CPU-compatible inference (no GPU required for deployment)
- Portable across operating systems (Windows, macOS, Linux)

Alignment with Objectives:

- Automated waste classification (6 categories)
 - Multi-object detection capability
 - Real-time processing
 - Practical application for waste segregation
-

Section 5: Challenges Faced

1. Dataset selection and Quality Issues: It was hard to find an appropriate dataset with quality images and of appropriate size. The original selected Roboflow dataset contained images with over 100 annotated objects, causing training instability and unrealistic learning scenarios.

2. Multi-object Detection Complexity: YOLO required learning complex grid-based prediction mechanisms and implementing Non-Maximum Suppression and a complex loss function.

3. Model Overfitting

Challenge: Initially trained versions showed high training accuracy but poor validation performance. **Solution:** Applied multiple regularization techniques:

- Dropout layers (0.5) in fully connected layers
- Early stopping with patience
- Batch normalization

4. Class Confusion: The model struggled to distinguish between visually similar classes, particularly Paper vs. Cardboard. Model accuracy was poor and couldn't be improved due to limited time.

5. Computational Resource Limitations: Limited access to GPU resources for extended hyperparameter tuning and experimentation.

Section 6: Secondary Goals and Future Scope

Secondary Goals:

1. Deploy to cloud platform for public accessibility
2. Add multiple image processing feature
3. Implement comprehensive evaluation metrics (mAP, precision)
1. Expand to 10+ waste categories (e-waste, textiles, hazardous materials)
2. Include location-specific disposal guidelines
3. Develop mobile application (iOS/Android)

Long-term Vision:

1. Video stream processing for industrial waste sorting lines
 2. Integration with robotic sorting systems
-

Section 7: References

1. [Roboflow Dataset for project](#)
 2. [Research paper for official YOLO architecture](#)
 3. [CNNs in pytorch](#)
 4. [YOLOv1 from scratch](#)
 5. [flask playlist](#)
 6. [writing CNNs from scratch using pytorch](#)
 7. [CNN from scratch, for troubleshooting](#)
 8. [similar YOLOv1 architecture project](#)
 9. [geeksforgeeks, to learn all ML concepts and loss concepts in depth](#)
 10. [pytorch tutorials](#)
 11. [pillow PIL and image handling in python](#)
 12. [to understand YOLOv8](#)
 13. [for YOLO models results evaluation](#)
 14. [for bounding box coordinates](#)
-

Section 8: Appendix

A. Alternative Approaches Considered

Pre-trained YOLOv11 Model

Approach: Fine-tuning YOLOv11s model on waste classification dataset

Not Selected because:

- Limited learning opportunity compared to building from scratch
- Goal was to understand YOLO fundamentals
- Received feedback from mentor

B. Non-Maximum Suppression Implementation

```
def nms(boxes, conf_thresh=0.2, iou_thresh=0.5):
    boxes = [b for b in boxes if b[1] > conf_thresh]
    boxes.sort(key=lambda x: x[1], reverse=True)
    result = []

    while boxes:
        chosen = boxes.pop(0)
        boxes = [b for b in boxes if b[0] != chosen[0] or
iou(chosen[2:], b[2:]) < iou_thresh]
        result.append(chosen)

    return result
```

C. Original YOLO v1 Architecture

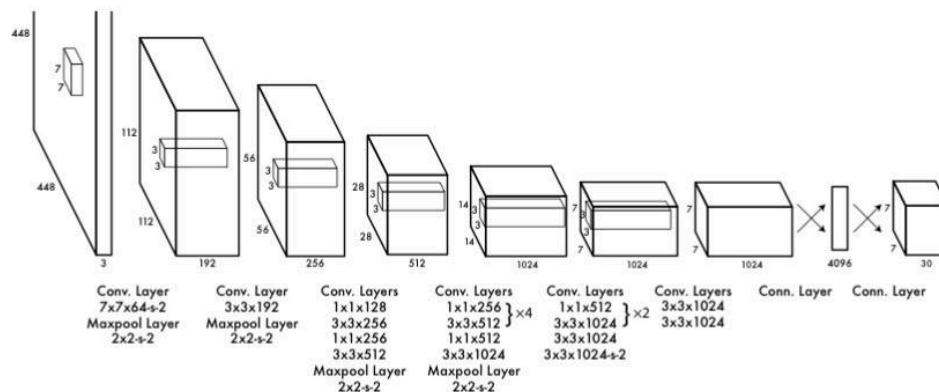


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

Image Reference: [Research paper for official YOLO architecture](#)

D. YOLO Loss:

loss function:

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad (3) \end{aligned}$$

Image Reference: [Research paper for official YOLO architecture](#)
